

Notas de clase
Unidad 0: Herramientas y preliminares
Eduardo Acuña Yeomans
Lenguajes de Programación
Semestre 2026-2
Lic. en Ciencias de la Computación
Universidad de Sonora

Cómo usar estas notas

Estas notas cubren las ocho sesiones de la unidad 0. No son un resumen de las diapositivas: son la versión escrita, con los ejemplos completos y con las cosas que en clase se dicen de viva voz y no quedan en ningún lado.

Cada sección corresponde a una sesión y lleva marcado su número. Puedes leerlas en orden, o abrir la que necesites: al final hay tres apéndices pensados para consultar a mitad de un problema: una referencia rápida de todo lo que la unidad usa, una lista de los errores de Racket que vas a encontrar con la traducción de qué significan, y un mapa de qué sección cubre qué.

Todo el código de estas notas está probado. Las sesiones de REPL transcritas muestran exactamente lo que Racket responde. Si tecleas algo y obtienes otra cosa, la diferencia importa: búscala.

Los ejercicios *no* están aquí. Van en hojas aparte, una por tema.

Qué se supone que ya sabes

De **Autómatas y Lenguajes Formales**: qué es una gramática libre de contexto, qué es una derivación y qué es un árbol de derivación. La unidad usa esas ideas más adelante, y no las vuelve a explicar desde cero. Lo que sí explica es la notación que usaremos y, sobre todo, qué tiene que ver una gramática con un programa de Racket.

De programación: haber escrito código en algún lenguaje. No se supone ninguna experiencia previa con Racket, Scheme o Lisp.

Lo que **no** se supone es que ya sepas programar sin variables que cambian de valor y sin ciclos. Esa es la parte incómoda de estas dos semanas, y hay una sección escrita para ella.

Índice

Parte I. Racket como lenguaje

5

1. Primer contacto: expresiones y valores	5
1.1. Instalarlo y trabajar con él	5
1.2. El REPL: leer, evaluar, imprimir	5
1.3. Notación prefija: el operador va primero	6
1.4. Todo es una expresión que produce un valor	6
1.5. Ponerle nombre a las cosas: <code>define</code>	7
1.6. Traducir del lenguaje que ya conoces	7
2. Recursión y listas	8
2.1. El problema	8
2.2. Lo mismo, sin ciclo	8
2.3. Qué pasa cuando lo corres	9
2.4. La receta	9
2.5. Dos más, del ciclo a la recursión	9
2.6. Cuando hay más de dos casos: <code>cond</code>	10
2.7. Listas	11
2.8. Una lista es un dato recursivo	12
2.9. Contar y construir: <code>length</code> y <code>append</code>	12
2.10. Dos funciones casi idénticas	13
2.11. Quedarse con algunos: <code>filter</code>	13
2.12. La función es un valor	14
3. <code>struct</code> y <code>match</code>	14
3.1. El problema: datos que vienen en variantes	14
3.2. <code>struct</code>	15
3.3. En el REPL	15
3.4. Qué ganamos	16
3.5. El área, primer intento	16
3.6. El área, con <code>match</code>	16
3.7. Cómo se lee un caso	17
3.8. Un caso por variante	17
3.9. Agregar una variante	17
Parte II. Programas como datos	18
4. De la gramática al dato	18
4.1. Puente desde Autómatas	18
4.2. La gramática con la que vamos a trabajar	19
4.3. Derivaciones	19
4.4. Dos representaciones del mismo programa	20
4.5. Tres textos, un solo árbol	20
4.6. Cada producción es un <code>struct</code>	21
4.7. Construir un árbol a mano	21
4.8. Lo que acabas de hacer	22

5. calc: el programa que evalúa programas	22
5.1. Lo que queremos	23
5.2. Empezar por lo fácil	23
5.3. add-exp: aquí está el punto	23
5.4. calc, completa	23
5.5. Qué pasa cuando lo corres	24
5.6. Lo que acaba de pasar	24
5.7. Agregar neg	25
5.8. La resta, camino 1	25
5.9. La resta, camino 2	25
5.10. Los dos caminos	26
6. Especificar y trazar	26
6.1. Calentamiento: una función definida por casos	26
6.2. Por qué la prosa no basta	27
6.3. La especificación de calc	27
6.4. Cómo se lee una regla	28
6.5. Lo mismo, dos veces	28
6.6. Qué fija cada una	28
6.7. Trazar: el formato	29
6.8. Un trazado completo	29
6.9. Otro, para comparar	29
Parte III. Interfaz e implementación y cierre	30
7. Interfaz e implementación	30
7.1. Interfaz e implementación	30
7.2. Módulos y contratos, en un minuto	31
7.3. Lectura adelantada: del texto al árbol	32
7.4. Lectura adelantada: pruebas con RackUnit	33
8. Nombres y with	33
8.1. La gramática completa	33
8.2. Qué hace with	34
8.3. Los structs que faltan	34
8.4. A calc le falta algo	34
8.5. Las asociaciones	35
8.6. with-exp	35
8.7. La especificación completa	36
8.8. Trazar con σ	36
A. Referencia rápida	36
B. Errores frecuentes y cómo leerlos	38

1. Primer contacto: expresiones y valores

Sesión 1

Racket es un lenguaje de la familia **Scheme**, que a su vez viene de Lisp. Lo usamos por una razón concreta: está diseñado, entre otras cosas, para construir lenguajes, que es de lo que trata el curso.

1.1. Instalarlo y trabajar con él

Descárgalo de <https://racket-lang.org>. Hay dos formas de trabajar:

- **DrRacket**, el entorno gráfico. La ventana está partida en dos: arriba el editor, donde escribes el programa y lo guardas, y abajo el REPL, donde pruebas cosas sueltas. El botón *Run* carga lo de arriba en el de abajo.
- **racket** en la terminal, que es solo el REPL.

Todo archivo de Racket empieza declarando en qué lenguaje está escrito. El nuestro siempre es el mismo:

```
#lang racket
```

1.2. El REPL: leer, evaluar, imprimir

REPL viene de *read-eval-print loop*: lee lo que escribes, lo evalúa, imprime el resultado, y vuelve a empezar.

```
> (+ 1 2)
3
> (* 3 4)
12
> (+ 1 (* 3 4))
13
```

El `>` lo pone Racket. Lo que sigue lo escribes tú. En estas notas todas las sesiones de REPL se muestran así: con `>` lo que se teclea, y debajo lo que Racket contesta.

Lo que escribes es una **expresión**. Lo que Racket devuelve es un **valor**. Esas dos palabras van a aparecer todo el semestre, así que conviene usarlas bien desde ahora.

1.3. Notación prefija: el operador va primero

En Racket el operador se escribe *antes* de sus argumentos, dentro del paréntesis:

En C++ o Python	En Racket
<code>1 + 2</code>	<code>(+ 1 2)</code>
<code>2 + 3 * 4</code>	<code>(+ 2 (* 3 4))</code>
<code>(2 + 3) * 4</code>	<code>(* (+ 2 3) 4)</code>
<code>max(a, b)</code>	<code>(max a b)</code>

Tres consecuencias que vale la pena hacer explícitas:

- **Un paréntesis casi siempre significa lo mismo:** mandar a llamar una función, donde lo primero es la función y el resto son sus argumentos. No hay paréntesis de agrupación por un lado y de llamada por otro. Las excepciones son pocas y tienen nombre: `define`, `if` y `cond` son **formas especiales**, y cada una decide por su cuenta qué evalúa y en qué orden.
- **No hay reglas de precedencia que recordar.** En `2 + 3 * 4` tienes que saber que `*` va antes. En `(+ 2 (* 3 4))` la estructura ya está escrita. La ambigüedad no existe porque no hay dónde ponerla.
- **+ y max no son cosas distintas.** En el lenguaje del que vienes, uno es un operador y el otro una función. Aquí los dos son funciones y se escriben igual.

Sugerencia: el error de los paréntesis de más

Viniendo de otros lenguajes es natural escribir un paréntesis extra “por claridad”: `((+ 1 2))`. Eso no agrupa: dice “evalúa `(+ 1 2)`, que da 3, y ahora aplica 3 como si fuera una función”. Racket contesta `application: not a procedure`. Ver el apéndice B.

1.4. Todo es una expresión que produce un valor

Esta es la idea de esta sección, y es la que más cuesta si vienes de C++. Ahí hay cosas que *hacen* y cosas que *valen*. Un `if` de C++ no vale nada: decide qué se ejecuta.

```
int y;  
if (x > 0)      // el if no produce nada:  
    y = 1;     // solo decide qué se ejecuta  
else  
    y = -1;
```

En Racket el `if` **produce un valor**, igual que cualquier otra expresión:

```
> (if (> x 0) 1 -1)
1
> (* 10 (if (> x 0) 1 -1))
10
```

La segunda línea es el punto: el `if` está *dentro* de una multiplicación. Si algo produce un valor, se puede poner donde quepa un valor. Eso vale para el `if` y para todo lo demás que veremos.

C++ tiene algo parecido, el operador `x > 0 ? 1 : -1`, que sí produce un valor. Pero es un operador aparte del `if` y solo sirve para elegir entre dos valores. En Racket no hacen falta dos construcciones, porque el `if` normal ya es una expresión.

1.5. Ponerle nombre a las cosas: `define`

Un nombre para un valor:

```
(define pi 3.1416)

> (* pi 2)
6.2832
```

Un nombre para una función:

```
(define (cuadrado x)
  (* x x))

> (cuadrado 7)
49
> (cuadrado (+ 1 2))
9
```

Fíjate en la forma: `(define (nombre parámetros...) cuerpo)`. El nombre de la función y sus parámetros van juntos entre paréntesis, con la misma forma que tendrá la llamada.

El cuerpo de la función es **una expresión**, y su valor es lo que la función devuelve. No se escribe `return`: no hace falta, porque no hay nada más que pueda ser el resultado.

1.6. Traducir del lenguaje que ya conoces

Cuatro traducciones resueltas:

Original

En Racket

`3 * (4 + 5)`

`(* 3 (+ 4 5))`

`(a + b) / 2`

`(/ (+ a b) 2)`

`int doble(int x) { return x * 2; }`

`abs(x)`, con un `if`

`(define (doble x) (* x 2))`

`(define (valor-abs x) (if (< x 0) (- x) x))`

En la última, `(- x)` con un solo argumento es el negativo de `x`. Con dos argumentos, `(- a b)`, es la resta. Las dos cosas se escriben con el mismo símbolo.

2. Recursión y listas

Sesión 2

Esta es la sección incómoda si vienes de programación imperativa, y la que conviene releer si algo no fluye después.

2.1. El problema

Así es como se escribe normalmente el factorial en C++:

```
int factorial(int n) {  
    int r = 1;  
    for (int i = 1; i <= n; i++)  
        r = r * i;  
    return r;  
}
```

En Racket **esto no se puede escribir**. No hay `for` y no hay `while`. Tampoco hay una variable `r` que vaya cambiando de valor.

No es una limitación que haya que rodear con un truco: es la forma normal de trabajar aquí. La repetición se hace con **recursión**, y es el único mecanismo de repetición que vamos a usar en todo el curso.

2.2. Lo mismo, sin ciclo

```
(define (factorial n)  
  (if (= n 0)  
      1  
      (* n (factorial (- n 1)))))
```

- **Caso base:** el factorial de 0 es 1. Ahí se acaba, sin calcular nada.
- **Caso recursivo:** el factorial de `n` se arma con el factorial de `n - 1`, que es un problema más chico.

- **No hay ninguna variable que cambie de valor.** Nada se modifica. El `n` de cada llamada es el suyo y no se toca.

2.3. Qué pasa cuando lo corres

Vale la pena ver el cálculo completo, porque es el mismo movimiento que más adelante se vuelve una técnica con nombre:

```
(factorial 4)
= (* 4 (factorial 3))
= (* 4 (* 3 (factorial 2)))
= (* 4 (* 3 (* 2 (factorial 1))))
= (* 4 (* 3 (* 2 (* 1 (factorial 0)))))
= (* 4 (* 3 (* 2 (* 1 1))))
= 24
```

Cada renglón reemplaza una llamada por lo que esa llamada vale. Nada se guarda en ningún lado: solo se sustituye. Si alguna vez te pierdes con una función recursiva, escribe esto en papel. Funciona siempre.

2.4. La receta

Para escribir una función recursiva, tres preguntas en este orden:

1. **¿Cuál es el caso en el que ya sé la respuesta sin calcular nada?** Ese es el caso base. En el factorial, `n = 0`.
2. **¿Cómo hago el problema más chico?** (`- n 1`) en el factorial.
3. **¿Cómo combino la respuesta del problema chico con lo mío?** (`* n ...`) en el factorial.

Si el problema no se hace más chico en cada llamada, no podemos garantizar fácilmente que la función termina de calcular.

2.5. Dos más, del ciclo a la recursión

Sumar de 1 a `n`:

```
(define (suma-hasta n)
  (if (= n 0)
      0
      (+ n (suma-hasta (- n 1)))))

> (suma-hasta 4)
10
```

Elevar b a la e :

```
(define (potencia b e)
  (if (= e 0)
      1
      (* b (potencia b (- e 1)))))

> (potencia 2 5)
32
```

Sugerencia: la traducción mecánica

En los tres casos el patrón es el mismo: **el acumulador del ciclo se vuelve el valor que devuelve la llamada**. Donde el ciclo escribía $r = r * i$, la versión recursiva escribe $(* n (factorial (- n 1)))$. Donde escribía $r = r + i$, escribe $(+ n (suma-hasta (- n 1)))$. El valor inicial del acumulador (1 para productos, 0 para sumas) es el caso base.

2.6. Cuando hay más de dos casos: cond

El if sirve para dos casos. Cuando son más, anidarlos se vuelve ilegible:

```
(define (signo n)
  (if (< n 0)
      -1
      (if (= n 0)
          0
          1)))
```

cond escribe lo mismo como una lista de casos:

```
(define (signo n)
  (cond [(< n 0) -1]
        [(= n 0) 0]
        [else 1]))

> (signo -7)
-1
> (signo 0)
0
```

Cada renglón tiene dos partes: una condición y qué vale si esa condición se cumple. Racket las prueba de arriba abajo y se queda con la primera que da verdadero. `else` es la que se cumple siempre, y va al final.

Los corchetes son paréntesis: en Racket `[` y `(` son intercambiables. Se usan corchetes por costumbre en los casos de `cond` (y más adelante en los de `match`) para que se distingan de un vistazo.

2.7. Listas

Una lista se construye con `cons`, o con `list` que es un atajo:

```
> (list 1 2 3)
'(1 2 3)
> (cons 1 (cons 2 (cons 3 '())))
'(1 2 3)
> '()
'()
```

- `'()` es la **lista vacía**.
- `cons` pega un elemento al frente de una lista y devuelve **una lista nueva**: la original no se toca. Esto no es un detalle, es la regla en todo lo que escribiremos.
- La comilla simple con la que Racket imprime `'(1 2 3)` forma parte de cómo se escribe una lista literal. Por ahora tómallo como parte de la notación.

Y se mira adentro con `car`, `cdr` y `null?`:

```
> (car '(1 2 3))
1
> (cdr '(1 2 3))
'(2 3)
> (null? '())
#t
> (null? '(1 2 3))
#f
```

- `car` devuelve el primer elemento. También puedes usar `first`.
- `cdr` devuelve el resto, *como lista*. Fíjate en que `(cdr '(1 2 3))` no es 2: es `'(2 3)`. También puedes usar `rest`.
- `null?` pregunta si la lista está vacía. Ese va a ser el caso base de casi todo. También puedes usar `empty?`.
- `#t` y `#f` son verdadero y falso.

Los nombres `car` y `cdr` son herencia del Lisp de 1958 y se refieren a registros de una computadora que ya no existe. No significan nada: son historia, y se quedaron.

2.8. Una lista es un dato recursivo

Aquí está la idea que sostiene el resto de la unidad. Mira cómo se define una lista:

Una lista es:

- la lista vacía '()', o
- (`cons e lst`), donde `e` es un elemento y `lst` es otra lista.

La definición tiene **dos casos**, y el segundo se define en términos de sí mismo. La función que la recorra va a tener los mismos dos casos:

```
(define (f lst)
  (if (null? lst)
      ... ; la lista vacía
      ... (car lst) ... (f (cdr lst)) ...))
```

Nota Didáctica: la frase que hay que guardar

La forma del dato dicta la forma de la función. Es la frase que más se va a repetir en estas dos semanas: aquí con listas, y eventualmente con programas. Cuando escribas una función que evalúa expresiones, va a tener un caso por cada variante de la gramática, por exactamente esta razón y no por casualidad.

2.9. Contar y construir: `length` y `append`

```
(define (longitud lst)
  (if (null? lst)
      0
      (+ 1 (longitud (cdr lst)))))
```

```
> (longitud '(a b c))
3
```

Caso base: la lista vacía tiene 0 elementos. Caso recursivo: uno más que los que tiene el resto. Es la receta del factorial con (`cdr lst`) en lugar de (`- n 1`).

```
(define (concatenar l1 l2)
  (if (null? l1)
      l2
      (cons (car l1) (concatenar (cdr l1) l2))))
```

```
> (concatenar '(1 2) '(3 4))
'(1 2 3 4)
```

Dos cosas cambian respecto de longitud: el caso base ya no es un número sino 12 completa, y el caso recursivo **construye** con **cons** en vez de sumar. Ni 11 ni 12 cambian: el resultado es una lista nueva.

Racket ya trae **length** y **append**. Las escribimos con otro nombre para ver qué hay adentro.

2.10. Dos funciones casi idénticas

Mira estas dos y busca la diferencia:

```
(define (duplicar-todos lst)
  (if (null? lst)
      '()
      (cons (* 2 (car lst)) (duplicar-todos (cdr lst)))))

(define (cuadrar-todos lst)
  (if (null? lst)
      '()
      (cons (cuadrado (car lst)) (cuadrar-todos (cdr lst)))))
```

Todo es igual menos una cosa: qué se le hace a cada elemento. Se puede escribir una sola función que reciba *la operación* como un argumento más:

```
(define (mapea f lst)
  (if (null? lst)
      '()
      (cons (f (car lst)) (mapea f (cdr lst)))))
```

Y se usa así:

```
> (mapea cuadrado '(1 2 3))
'(1 4 9)
> (mapea factorial '(3 4 5))
'(6 24 120)
```

2.11. Quedarse con algunos: filter

El mismo movimiento, pero en lugar de transformar cada elemento, decidir cuáles se quedan:

```
(define (filtra p lst)
  (cond [(null? lst) '()]
        [(p (car lst)) (cons (car lst) (filtra p (cdr lst)))]
        [else (filtra p (cdr lst))]))

> (filtra even? '(1 2 3 4 5 6))
'(2 4 6)
```

Tres casos, y por eso `cond` en lugar de `if`: la lista está vacía, el primer elemento pasa la prueba y se conserva, o el primer elemento no pasa y se descarta. En los dos últimos la llamada recursiva es la misma. Lo único que cambia es si se hace `cons` del elemento o no.

`p` es una función que devuelve verdadero o falso. `even?` es una que ya trae Racket.

2.12. La función es un valor

Fíjate en lo que acabas de hacer en `mapea` y `filtra`: pasaste `cuadrado`, `factorial` y `even?` como argumentos.

- `cuadrado` entra como argumento igual que entraría un número.
- No se escribe (`cuadrado`) ni nada especial: se pasa el nombre y ya. Los paréntesis son para *llamarla*, y aquí no la estamos llamando, sino entregando.
- En Racket una función es un valor como cualquier otro: se puede pasar como argumento, devolver como resultado y guardar en una estructura.

Nota Didáctica: hasta dónde llega esto por ahora

Basta con que lo veas funcionar y sepas escribirlo. Lo que *significa* que una función sea un valor (y las consecuencias que tiene, que son grandes) es materia de la unidad 2, cuando el lenguaje que estemos construyendo tenga procedimientos.

3. `struct` y `match`

Sesión 3

Las dos herramientas de esta sección son las que vamos a usar todos los días de aquí a diciembre.

3.1. El problema: datos que vienen en variantes

Queremos calcular el área de una figura. Un círculo se describe con un radio, y un rectángulo con base y altura. Son la misma clase de cosa y no se guardan igual.

Con listas, que es lo único que tenemos hasta ahora:

```
(define c (list 'circulo 3))  
(define r (list 'rectangulo 2 5))
```

Funciona, y tiene tres problemas:

- ¿Cuál de los dos números de `r` es la base? Hay que acordarse.
- `(car (cdr r))` no dice nada sobre lo que saca. Quien lea el código tiene que reconstruir mentalmente qué había en esa posición.
- Nada impide escribir `(list 'circulo)` y quedarse sin radio. El error no aparece hasta mucho después, en otro lado.

3.2. struct

```
(struct circulo (radio) #:transparent)
(struct rectangulo (base altura) #:transparent)
```

Cada línea define un tipo de dato con campos con nombre. Cada una te da **tres cosas** de golpe:

Qué	Cómo se llama	Para qué
constructor	circulo, rectangulo	crear uno
selectores	circulo-radio, rectangulo-base, rectangulo-altura	sacar un campo
predicado	circulo?, rectangulo?	preguntar de qué variante es

El nombre de cada selector se arma solo: el del struct, un guion, y el del campo.

`#:transparent` sirve para que el REPL nos deje ver lo que hay adentro. Sin él, Racket imprime `#<circulo>` y nada más, lo cual hace difícil la tarea de depurar. Ponlo siempre.

3.3. En el REPL

```
> (define c (circulo 3))
> c
(circulo 3)
> (circulo-radio c)
3
> (circulo? c)
#t
> (rectangulo? c)
#f
> (circulo 3 4)
circulo: arity mismatch;
the expected number of arguments does not match the given number
  expected: 1
  given: 2
```

Ese último error es una buena noticia: un círculo tiene exactamente un campo, y Racket lo sabe. Con la lista de antes, `(list 'circulo 3 4)` habría pasado sin protestar.

3.4. Qué ganamos

Con listas	Con struct
<code>(list 'rectangulo 2 5)</code>	<code>(rectangulo 2 5)</code>
<code>(car (cdr r))</code>	<code>(rectangulo-base r)</code>
“¿era base o altura?”	el nombre lo dice
un campo de menos: pasa	un campo de menos: error

Un `struct` no guarda más información que la lista. Guarda la misma, pero **con nombre y con forma fija**.

3.5. El área, primer intento

Queremos una sola función que sirva para las dos variantes. Con lo que tenemos:

```
(define (area f)
  (cond [(circulo? f) (* 3.1416 (circulo-radio f) (circulo-radio f))]
        [(rectangulo? f) (* (rectangulo-base f) (rectangulo-altura f))]
        [else (error "no es una figura")]))
```

Tres casos, así que `cond` y no `if` anidados. Pregunta de qué variante es, y luego saca los campos uno por uno. Con dos variantes se lee. Con cuatro es ilegible.

3.6. El área, con `match`

```
(define (area f)
  (match f
    [(circulo r) (* 3.1416 r r)]
    [(rectangulo b h) (* b h)]))
```

```
> (area (circulo 3))
28.2744
> (area (rectangulo 2 5))
10
```

- Un caso por variante, y ya.
- `match` hace las dos cosas de golpe: reconoce de qué variante es y le pone nombre a los campos.
- No aparece ningún selector. No hacen falta.

Habrás notado que se perdió algo por el camino: la versión con `cond` tenía un `else` que llamaba a `error`, y esta no tiene nada parecido. ¿Entonces qué contesta Racket si le pasas algo que no es un círculo ni un rectángulo? Vale la pena teclearlo antes de seguir leyendo:

```
> (area 5)
```

Lo que sale está en el apéndice B, junto con los demás errores de la unidad.

En `#lang racket`, `match` ya viene. Si algún día trabajas en `#lang racket/base`, hay que pedirlo con `(require racket/match)`.

3.7. Cómo se lee un caso

```
[(circulo r) (* 3.1416 r r)]
```

- A la izquierda, el **patrón**. A la derecha, qué hacer si el patrón aplica.
- `(circulo r)` *parece* una llamada al constructor, pero aquí no construye nada: **desarma**. Es la misma forma escrita, usada al revés.
- **r no es una variable que ya exista**. Es un nombre nuevo que `match` le pone al campo, y que solo vive dentro de ese caso. Si afuera hay otra cosa llamada `r`, no tiene nada que ver con esta.
- Si un campo no te interesa, se ignora con el comodín `_`: `[(rectangulo b _) b]`.

3.8. Un caso por variante

El dato tiene...	...la función tiene
la lista vacía <i>o</i> un <code>cons</code>	dos casos
un círculo <i>o</i> un rectángulo	dos casos
tres variantes	tres casos

Es la frase de la sección anterior otra vez: la forma del dato dicta la forma de la función. Con listas primero, ahora con figuras, y más adelante con programas.

3.9. Agregar una variante

Un cuadrado se describe con un lado. Se define el `struct` y se extiende `area`:

```
(struct cuadrado (lado) #:transparent)

(define (area f)
  (match f
    [(circulo r) (* 3.1416 r r)]
    [(rectangulo b h) (* b h)]
    [(cuadrado l) (* l l)]))
```

```
> (area (cuadrado 4))
```

16

Son **dos** cambios, y hay que hacer los dos. Si defines el **struct** y olvidas el caso:

```
> (area (cuadrado 4))
```

```
match: no matching clause for (cuadrado 4)
```

El error sale **cuando corres**, no antes: nadie te avisa al escribirlo. Por eso “un caso por variante” no es cuestión de estilo: es lo que hace que la función esté **completa**.

Nota Didáctica: una pregunta que dejamos abierta

¿Hacía falta una variante nueva, o un cuadrado es un rectángulo con la base igual a la altura? Las dos respuestas se pueden defender, y la decisión tiene consecuencias. Más adelante vas a ver la misma disyuntiva otra vez, con la resta, y tampoco la vamos a cerrar ahí: es una de las preguntas que el curso deja abiertas a propósito.

Parte II · Programas como datos

4. De la gramática al dato

Sesión 4

Nada de esta sección es nuevo del todo: las gramáticas ya las viste en Autómatas. Lo nuevo son dos cosas: la notación que usaremos, y qué tiene que ver una producción de una gramática con un **struct** de Racket.

4.1. Puente desde Autómatas

Es la misma idea de siempre (reglas que dicen qué cadenas pertenecen al lenguaje) con otra notación:

En Autómatas

$$\begin{array}{lcl} E & \rightarrow & E + T \\ & | & T \\ T & \rightarrow & \text{num} \end{array}$$

En la notación del curso

$$\begin{array}{lcl} \text{Exp} & ::= & \text{Exp} + \text{Term} \\ & | & \text{Term} \\ \text{Term} & ::= & \text{Num} \end{array}$$

La notación de la derecha tiene nombre propio: se llama **BNF**, por *Backus-Naur Form*, la forma de Backus y Naur. Es la notación con la que se describió la sintaxis de ALGOL 60 a

principios de los sesenta, y desde entonces es la manera estándar de escribir una gramática libre de contexto. Cuando en este curso digamos “la BNF del lenguaje”, nos referimos a esto: la lista de producciones que dicen qué se puede escribir.

Las convenciones, que son las del libro de texto (EOPL §1.1.2):

- `::=` es la flecha. Se lee “*es*” o “*puede ser*”.
- Los no-terminales se escriben con inicial mayúscula.
- El lado izquierdo se omite cuando se repite: por eso las líneas que empiezan con `|`.

4.2. La gramática con la que vamos a trabajar

```
Exp ::= Num
      | add(Exp, Exp)
      | mul(Exp, Exp)
```

Tres producciones. Como toda gramática, define exactamente qué cadenas pertenecen al lenguaje y cuáles no:

Sí las genera	No las genera
3	add(3)
add(3, 4)	add 3 4
add(3, mul(2, 4))	add(3, 4, 5)
mul(add(1, 2), add(3, 4))	sub(3, 4)

Las de la derecha fallan por razones distintas: a la primera le falta un argumento, la segunda no tiene la forma `add(...)`, la tercera tiene uno de sobra, y la cuarta usa una palabra que la gramática no menciona.

4.3. Derivaciones

Para convencerte de que una cadena pertenece al lenguaje, la derivas: empiezas por el no-terminal inicial y vas reemplazando, un paso a la vez.

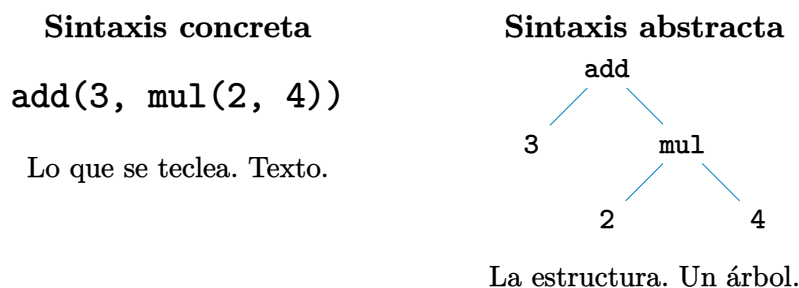
```
Exp
⇒ add(Exp, Exp)
⇒ add(Num, Exp)
⇒ add(3, Exp)
⇒ add(3, mul(Exp, Exp))
⇒ add(3, mul(Num, Exp))
⇒ add(3, mul(2, Exp))
⇒ add(3, mul(2, Num))
⇒ add(3, mul(2, 4))
```

Cada paso reemplaza **un** símbolo, y solo uno. Cuando ya no queda ningún no-terminal, la derivación terminó y lo que queda es una cadena del lenguaje.

Los pasos que cambian **Num** por un número concreto van aparte de los que aplican **Exp ::= Num**, aunque se sienta lento. **Num** es el único símbolo de esta gramática sin producción propia: representa cualquier número, y por eso no hay una línea de la gramática a la que apuntar cuando lo reemplazas.

Con `add(mul(2), 4)` el intento se atora: para poner `mul` hace falta la producción `mul(Exp, Exp)`, que exige dos expresiones, y ahí solo hay una. No hay derivación, y por lo tanto la cadena no pertenece al lenguaje.

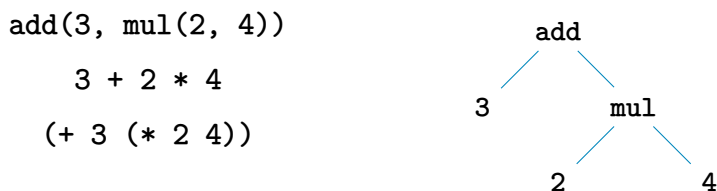
4.4. Dos representaciones del mismo programa



En el texto hay comas, paréntesis y espacios. En el árbol no queda nada de eso: solo qué operación es y cuáles son sus partes. Las dos son representaciones del mismo programa, y sirven para cosas distintas: el texto para escribirlo, el árbol para procesarlo.

Esta distinción es de EOPL §2.5.

4.5. Tres textos, un solo árbol



La notación del curso, la de C++ y la de Racket. Tres formas de escribirlo: la misma estructura, y el mismo resultado.

- Cómo se escribe un programa es una decisión de **notación**. El árbol es lo que de verdad se procesa.
- Pasar del texto al árbol tiene nombre y ya lo conoces: **análisis sintáctico**. Más adelante en el curso lo automatizamos.

Nota Didáctica: una pregunta que nos haremos todo el curso

¿En qué se distingue cambiar *cómo se escribe* un programa de cambiar *lo que hace*? Suena obvio y no lo es: buena parte de lo que discutiremos de aquí a diciembre depende de trazar esa línea con precisión. Por ahora quédate con que las tres cadenas de arriba se escriben distinto y significan lo mismo.

4.6. Cada producción es un struct

Aquí está el paso que convierte una gramática en código:

Producción	Struct
Num	(<code>struct num-exp (n)</code>)
add(Exp, Exp)	(<code>struct add-exp (izq der)</code>)
mul(Exp, Exp)	(<code>struct mul-exp (izq der)</code>)

Usamos dos reglas para convertir una gramática en código:

- Una producción → un `struct`.
- Cada `Exp` del lado derecho → un campo.

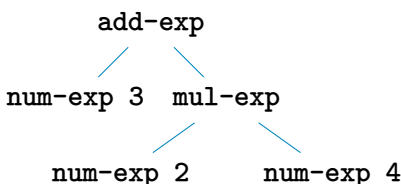
Es exactamente lo de la sección 3 con las figuras. Lo único que cambió es de dónde salen las variantes: ahora salen de una gramática.

```
(struct num-exp (n) #:transparent)  
(struct add-exp (izq der) #:transparent)  
(struct mul-exp (izq der) #:transparent)
```

Con una diferencia importante respecto de las figuras: los campos de `add-exp` y `mul-exp` son *a su vez expresiones*. Es un dato definido en términos de sí mismo, lo mismo que la lista de la sección 2, con tres variantes en lugar de dos. Por eso las funciones que lo recorran van a ser recursivas.

4.7. Construir un árbol a mano

```
> (define p (add-exp (num-exp 3)  
    (mul-exp (num-exp 2) (num-exp 4))))  
> p  
(add-exp (num-exp 3) (mul-exp (num-exp 2) (num-exp 4)))
```



La anidación del código es la forma del árbol. Lo que en el papel son hijos, en Racket son campos. Si te cuesta escribir uno de estos, dibuja primero el árbol y después transcríbelo de arriba hacia abajo.

Y para agregar una operación, la receta no cambia. Si queremos escribir `neg(add(1, 2))`, que representa el negativo de la expresión que lleva adentro:

```

; Exp ::= ... / neg(Exp)

(struct neg-exp (cuerpo) #:transparent)

> (neg-exp (add-exp (num-exp 1) (num-exp 2)))
(neg-exp (add-exp (num-exp 1) (num-exp 2)))

```

Una producción con un solo `Exp` del lado derecho: un solo campo.

4.8. Lo que acabas de hacer

Ese valor guardado en `p` es un **programa**.

- No lo ejecutaste. Lo **construiste**, y ahí está, guardado en una variable.
- Se puede guardar en una lista, pasar a una función, devolver como resultado. Es un dato como cualquier otro.
- `(add-exp-izq p)` te devuelve un pedazo del programa.

Nota Didáctica: lo que acabas de conseguir

Si un programa es solo un dato, entonces por sí mismo no significa nada. Alguien tiene que decir qué significa, y ese alguien va a ser **otro programa**. Eso es lo que escribimos en la sección siguiente, y en versiones cada vez más ambiciosas, es lo que hacemos durante todo el semestre.

5. calc: el programa que evalúa programas

Sesión 5

5.1. Lo que queremos

```
> (define p (add-exp (num-exp 3)
                      (mul-exp (num-exp 2) (num-exp 4))))
> (calc p)
11
```

`calc` recibe una **Exp** y devuelve un **número**. Ya tenemos todo lo que hace falta para escribirla: `match` y recursión.

5.2. Empezar por lo fácil

```
(define (calc e)
  (match e
    [(num-exp n) n]))

> (calc (num-exp 5))
5
```

El número ya venía adentro del nodo. Aquí no hay nada que calcular: solo hay que sacarlo.

5.3. `add-exp`: aquí está el punto

```
(define (calc e)
  (match e
    [(num-exp n) n]
    [(add-exp izq der) (+ (calc izq) (calc der))]))
```

Detente en este caso, porque es donde suele atorarse todo el mundo:

- **izq y der no son números**: son expresiones. Son nodos del árbol. `(+ izq der)` no tendría sentido: estarías intentando sumar dos árboles con una operación numérica.
- Para sumar necesito el *valor* de cada lado. ¿Y quién sabe calcular el valor de una expresión? `calc`, que es justo la función que estamos escribiendo.
- La función es recursiva porque el dato lo es. Otra vez lo mismo.

5.4. `calc`, completa

```
(define (calc e)
  (match e
    [(num-exp n) n]
    [(add-exp izq der) (+ (calc izq) (calc der))]
    [(mul-exp izq der) (* (calc izq) (calc der))]))
```

Tres producciones en la gramática, tres **structs**, tres casos aquí. Cada caso hace una sola cosa, y la hace para su variante. Nueve líneas: eso es todo el intérprete.

5.5. Qué pasa cuando lo corres

```
(calc p)
= (+ (calc (num-exp 3)) (calc (mul-exp (num-exp 2) (num-exp 4))))
= (+ 3 (calc (mul-exp (num-exp 2) (num-exp 4))))
= (+ 3 (* (calc (num-exp 2)) (calc (num-exp 4))))
= (+ 3 (* 2 4))
= (+ 3 8)
= 11
```

Cada paso aplica el caso que le toca a la variante que tiene enfrente. Es el mismo movimiento que hiciste con el factorial en la sección 2, y más adelante se vuelve una técnica con reglas propias.

5.6. Lo que acaba de pasar

El valor del todo se arma con los valores de las partes.

- El caso de **add-exp** no sabe si está en la raíz del árbol o quince niveles adentro. Solo mira sus dos hijos.
- Ningún caso necesita saber qué hay en el resto del programa.
- Por eso agregar una variante nueva no rompe las demás.

Acabas de escribir un programa cuya entrada es otro programa. Es de lo que va el resto del semestre.



5.7. Agregar neg

El `struct` ya lo definimos en la sección anterior. Falta el caso:

```
[(neg-exp cuerpo) (- (calc cuerpo))]  
  
> (calc (neg-exp (add-exp (num-exp 1) (num-exp 2))))  
-3
```

Un campo, una llamada recursiva. La receta de siempre.

5.8. La resta, camino 1

Ahora queremos restar. Lo obvio es una variante más y un caso más:

```
(struct sub-exp (izq der) #:transparent)  
  
(define (calc e)  
  (match e  
    [(num-exp n) n]  
    [(add-exp izq der) (+ (calc izq) (calc der))]  
    [(mul-exp izq der) (* (calc izq) (calc der))]  
    [(neg-exp cuerpo) (- (calc cuerpo))]  
    [(sub-exp izq der) (- (calc izq) (calc der))]))
```

Funciona. Y `calc` creció otra vez.

5.9. La resta, camino 2

Hay otra salida. Restar `b` es sumar el negativo de `b`, y las dos cosas ya las sabemos hacer:

```
(define (sub-exp a b)  
  (add-exp a (neg-exp b)))  
  
> (sub-exp (num-exp 5) (num-exp 2))  
(add-exp (num-exp 5) (neg-exp (num-exp 2)))  
> (calc (sub-exp (num-exp 5) (num-exp 2)))  
3
```

Fíjate bien en lo que pasó. Aquí `sub-exp` **no es un struct**: es una función que arma un árbol con las piezas que ya existían. Cuando la llamas, lo que obtienes es un `add-exp` con un `neg-exp` adentro.

Y por eso `calc` **no cambió ni una línea**. Nunca se enteró de que existe la resta.

5.10. Los dos caminos

Agregar un caso	Traducir a lo que ya hay
un <code>struct</code> nuevo	ningún <code>struct</code> nuevo
un caso nuevo en <code>calc</code>	<code>calc</code> no se toca
la resta es una operación más del lenguaje	la resta es una <i>forma de escribir</i> algo que el lenguaje ya tenía

Los dos funcionan. Los dos dan 3.

Nota Didáctica: la pregunta de Steele

Es la misma pregunta de la lectura de Steele: **¿qué tiene que ser primitivo en un lenguaje, y qué se puede definir después?** Cada vez que agregas algo a un lenguaje puedes elegir entre estos dos caminos, y la elección tiene consecuencias en cuánto código hay que mantener, en qué tan fácil es razonar sobre el lenguaje y en qué tan bien se comportan los mensajes de error.

No la vamos a contestar aquí. Vuelve en la unidad 1 y no se va en todo el curso.

6. Especificar y trazar

Sesión 6

Si hay una sección de esta unidad que conviene releer, es esta. Lo que se aprende aquí es la notación con la que el curso entero dice qué hace un lenguaje, y el formato de trazado que el ejercicio integrador califica.

6.1. Calentamiento: una función definida por casos

Antes de las expresiones, algo que ya conoces. Considera esta definición:

$$\begin{aligned}f(0) &= 1 \\f(n) &= n \times f(n-1) \quad \text{para } n > 0\end{aligned}$$

Son dos reglas. Para calcular $f(4)$, en cada paso se usa la que aplica y se escribe el resultado de aplicarla:

$$\begin{aligned}f(4) &= 4 \times f(3) \\&= 4 \times (3 \times f(2)) \\&= 4 \times (3 \times (2 \times f(1))) \\&= 4 \times (3 \times (2 \times (1 \times f(0)))) \\&= 4 \times (3 \times (2 \times (1 \times 1))) \\&= 24\end{aligned}$$

Otra, para comparar:

$$\begin{aligned}g(0) &= 0 \\g(n) &= n + g(n-1) \quad \text{para } n > 0 \\g(4) &= 4 + g(3) = 4 + (3 + g(2)) = 4 + (3 + (2 + g(1))) \\&= 4 + (3 + (2 + (1 + g(0)))) = 4 + (3 + (2 + (1 + 0))) = 10\end{aligned}$$

Si puedes escribir esos pasos intermedios, ya sabes trazar. Lo que sigue es lo mismo con otras reglas.

6.2. Por qué la prosa no basta

Intenta explicar qué hace `calc` sin enseñar el código:

“`calc` toma una expresión y devuelve un número. Si la expresión es un número, devuelve ese número. Si es una suma, devuelve la suma de los valores de sus dos partes. Si es un producto, el producto de los valores de sus dos partes. Si es una negación...”

Tres problemas, y ninguno se arregla escribiendo mejor:

- Se alarga y se repite.
- “El valor de sus partes”, ¿qué es el valor? Es justo lo que estamos definiendo. La frase se apoya en la palabra que todavía no significa nada.
- Con una duda concreta (¿qué pasa si una suma tiene adentro una negación?) hay que releer el párrafo entero y reconstruir la respuesta.

6.3. La especificación de `calc`

Lo mismo, en cuatro renglones:

$$\begin{aligned}[\text{num}] \quad & (\text{calc } (\text{num-exp } n)) = n \\[\text{add}] \quad & (\text{calc } (\text{add-exp } e_1 \ e_2)) = (\text{calc } e_1) + (\text{calc } e_2) \\[\text{mul}] \quad & (\text{calc } (\text{mul-exp } e_1 \ e_2)) = (\text{calc } e_1) \times (\text{calc } e_2) \\[\text{neg}] \quad & (\text{calc } (\text{neg-exp } e)) = -(\text{calc } e)\end{aligned}$$

Eso es todo, y no hay nada ambiguo. Cada regla lleva un nombre entre corchetes para poder citarla: los vas a usar en cuanto empieces a trazar.

Es importante tener a la mano la especificación sintáctica cuando vamos a trabajar con la semántica de un lenguaje. Las reglas de arriba dicen qué vale cada variante, y para saber qué variantes hay tienes la gramática de la sección 4. Léelas juntas.

6.4. Cómo se lee una regla

$$(\text{calc } (\text{add-exp } e_1 \ e_2)) = (\text{calc } e_1) + (\text{calc } e_2)$$

- A la izquierda, **qué forma tiene la entrada**. A la derecha, **cuánto vale**.
- Lo que está en *cursiva* es un hueco: e_1 y e_2 valen para cualquier expresión. Lo que está en monoespaciado es literal, y tiene que aparecer tal cual.
- El signo $+$ de la derecha es la suma de siempre, la de los números. **No es parte del lenguaje que estamos definiendo**: es la operación que usamos para definirlo. Esa distinción parece pedante ahora y va a ser central en la unidad 1.
- La regla se aplica únicamente cuando la entrada tiene esa forma.

6.5. Lo mismo, dos veces

Especificación

```
(calc (num-exp n))  
= n  
  
(calc (add-exp e1 e2))  
= (calc e1) + (calc e2)  
  
(calc (mul-exp e1 e2))  
= (calc e1) × (calc e2)
```

Código

```
(define (calc e)  
  (match e  
    [(num-exp n)  
     n]  
    [(add-exp e1 e2)  
     (+ (calc e1) (calc e2))]  
    [(mul-exp e1 e2)  
     (* (calc e1) (calc e2))]))
```

Cada cláusula del `match` es una ecuación. En el mismo orden, con los mismos huecos. Si escribes primero la especificación, el código sale casi solo, y esa es justamente la forma de trabajar que el curso propone.

6.6. Qué fija cada una

La especificación no dice

En qué lenguaje se implementa.
Si se calcula primero el lado izquierdo o el derecho.
Qué estructura de datos se usa.

El código sí decide todo eso

Racket, `match`, `struct`.
Un orden concreto de evaluación.
Y además **corre**.

Escribir la especificación primero es decidir **qué** queremos antes de decidir **cómo** lo hacemos. Cuando en el curso aparezcan dos implementaciones distintas de la misma cosa (y van a aparecer), la especificación es lo que permite decir que las dos son correctas.

La notación es la de EOPL §3.1–3.2, con una diferencia deliberada: allá los pasos de un cálculo no se etiquetan, y aquí sí. Ver la subsección siguiente.

6.7. Trazar: el formato

Trazar es calcular a mano, escribiendo cada paso. Tres reglas del juego:

1. Cada paso reemplaza una llamada a `calc` por el lado derecho de la regla que le toca.
2. Al lado de cada paso se anota **qué regla** se aplicó. Una sola regla por renglón: no se juntan dos reglas **distintas** en el mismo paso.
3. Las sustituciones se escriben. No se hacen de cabeza.

Este formato no es un capricho: es el que se pide en el ejercicio integrador, y es el que hace que un trazado se pueda revisar, tuyo o de alguien más.

6.8. Un trazado completo

Trazar `add(mul(2, 3), 4)`:

```
(calc (add-exp (mul-exp (num-exp 2) (num-exp 3)) (num-exp 4)))
= (calc (mul-exp (num-exp 2) (num-exp 3))) + (calc (num-exp 4))      [add]
= ((calc (num-exp 2)) × (calc (num-exp 3))) + (calc (num-exp 4))    [mul]
= (2 × (calc (num-exp 3))) + (calc (num-exp 4))                    [num]
= (2 × 3) + (calc (num-exp 4))                                     [num]
= 6 + (calc (num-exp 4))
= 6 + 4                                                            [num]
= 10
```

Los pasos que no llevan regla al lado son aritmética ($2 \times 3 = 6$), no son del lenguaje que estamos definiendo. Por eso no citan ninguna regla: no hay ninguna que citar.

6.9. Otro, para comparar

Trazar `mul(add(1, 2), add(3, 4))`:

```
(calc (mul-exp (add-exp (num-exp 1) (num-exp 2))
                (add-exp (num-exp 3) (num-exp 4))))
= (calc (add-exp (num-exp 1) (num-exp 2))) × (calc (add-exp (num-exp 3) (num-exp 4))) [mul]
= ((calc (num-exp 1)) + (calc (num-exp 2))) × (calc (add-exp (num-exp 3) (num-exp 4))) [add]
= (1 + 2) × (calc (add-exp (num-exp 3) (num-exp 4))) [num]
= 3 × ((calc (num-exp 3)) + (calc (num-exp 4))) [add]
= 3 × (3 + 4) [num]
= 21
```

Sugerencia: cómo verificar tu propio trazado

Corre la misma expresión en el REPL. Si el REPL da otra cosa que tu trazado, uno de los dos está mal, y vale mucho la pena averiguar cuál, porque las dos posibilidades enseñan algo distinto: o te equivocaste aplicando una regla, o entendiste mal qué hace el código.

7. Interfaz e implementación

Sesión 7

Esta sección es distinta de las demás. Lo que se trabaja en clase son las dos primeras subsecciones: la distinción entre interfaz e implementación, y la mención de módulos y contratos que la cierra.

Las dos últimas quedan aquí como **lectura adelantada**. El paso del texto al árbol y las pruebas automáticas se trabajan más adelante en el curso, cuando haya un intérprete de verdad al que aplicarlas. Léelas para saber qué existe y qué resuelve cada una. No hace falta que sepas escribirlas.

Nada de esta sección entra en el ejercicio integrador.

7.1. Interfaz e implementación

Supongamos que quiero contar cosas. ¿Qué necesita saber hacer un contador?

<code>(contador-nuevo)</code>	<i>; crea un contador con valor 0</i>
<code>(contador-inc c)</code>	<i>; regresa un contador con valor uno más que el de c</i>
<code>(contador-valor c)</code>	<i>; regresa el valor del contador c</i>

Esas tres operaciones son **la interfaz**: todo lo que quien lo usa necesita saber. Ahora, dos formas de construirlo:

Un número

```
(define (contador-nuevo)
  0)

(define (contador-inc c)
  (+ c 1))

(define (contador-valor c)
  c)
```

Una lista de unos

```
(define (contador-nuevo)
  '())

(define (contador-inc c)
  (cons 1 c))

(define (contador-valor c)
  (length c))
```

Distintas por dentro, las mismas tres operaciones por fuera. Y el código que las usa es idéntico:

```
(define (contar-hasta n)
  (if (= n 0)
      (contador-nuevo)
```

```
(contador-inc (contar-hasta (- n 1))))  
> (contador-valor (contar-hasta 5))  
5
```

Esto corre igual con las dos implementaciones. Ni una línea cambia, ni siquiera hay que releerlo.

La interfaz

Qué operaciones hay y qué prometen. Lo que el cliente necesita.

La implementación

Cómo están hechas por dentro. Lo que el cliente **no** necesita.

- Mientras la interfaz se respete, la representación se puede cambiar sin avisarle a nadie.
- El cliente que escribe `(+ c 1)` en lugar de `(contador-inc c)` se rompe en cuanto cambias de implementación, aunque hoy le funcione.

Tres preguntas para comprobar que la distinción quedó clara:

1. Cambio la lista de unos por una lista de ceros. ¿Se rompe algo?
2. Cambio `contador-valor` para que devuelva el doble. ¿Es un cambio de implementación o de interfaz?
3. Alguien escribió `(length c)` en su código en lugar de `(contador-valor c)` y le funcionó. ¿Por qué es un problema?

7.2. Módulos y contratos, en un minuto

Un módulo decide qué se ve desde afuera:

```
(provide contador-nuevo contador-inc contador-valor)
```

Y un contrato dice qué se espera de cada operación:

```
(provide (contract-out  
  [contador-nuevo (-> any/c)]  
  [contador-inc   (-> any/c any/c)]  
  [contador-valor (-> any/c exact-nonnegative-integer?)]))
```

Que los dos primeros digan `any/c` donde va el contador no es descuido. Es que el contrato tampoco puede nombrar la representación: si dijera “un número” o “una lista” estaría deshaciendo justo lo que la subsección anterior acaba de conseguir. Del `contador-valor` sí se puede prometer algo concreto, porque devuelve un número y eso es parte de la interfaz.

- **provide** es la interfaz del contador, pero escrita en el código y verificada por Racket: lo que no se declara, no se puede usar desde otro archivo.
- El contrato la hace verificable. Racket revisa en la frontera del módulo, y cuando algo falla dice quién tuvo la culpa: el que llamó o el que implementó.
- Los vas a ver en el código que se te entrega. No hay que escribirlos todavía.

Con esto se cierra lo que se trabaja en clase. Las dos subsecciones que siguen son las que quedaron como lectura adelantada: están aquí para que sepas qué existe y qué resuelve cada una, y se trabajan más adelante en el curso.

7.3. Lectura adelantada: del texto al árbol

Hasta ahora, para trabajar con `add(3, mul(2, 4))` hemos tecleado esto:

```
(add-exp (num-exp 3) (mul-exp (num-exp 2) (num-exp 4)))
```

Nadie programa así. Los programas se escriben como **texto**. Falta el paso que va del texto al árbol, y ya tiene nombre desde la sección 4: análisis sintáctico. Son dos etapas:

$$\text{"add(3, mul(2, 4))"} \xrightarrow{\text{lexer}} \text{tokens} \xrightarrow{\text{parser}} \text{AST}$$

- El **lexer** parte el texto en piezas: la palabra `add`, un paréntesis, el número 3, una coma... Cada pieza es un *token*.
- El **parser** arma esas piezas según la gramática y construye los **structs**.
- La herramienta **parser-tools** genera los dos a partir de una descripción. No se escriben a mano.

Lo que hay que retener es el resultado. Así se ve, aunque todavía no puedas teclearlo: el archivo con el lexer y el parser se entrega más adelante en el curso, junto con la explicación de cómo está hecho.

```
> (parse "add(3, mul(2, 4))")
(add-exp (num-exp 3) (mul-exp (num-exp 2) (num-exp 4)))
```

Devuelve exactamente el mismo struct que estabas tecleando. El árbol no cambia. Lo único que cambia es quién lo construye.

7.4. Lectura adelantada: pruebas con RackUnit

Hasta aquí, para comprobar que `calc` funciona has tecleado un caso en el REPL, mirado el resultado y confiado en tu memoria. Eso tiene tres problemas: no queda registro de qué se probó, no se puede repetir sin volver a teclear todo, y cuando toques `calc` para agregar un caso no vas a saber si rompiste algo que ya servía.

```
(require rackunit)

(check-equal? (calc (num-exp 5)) 5)
(check-equal? (calc (add-exp (num-exp 3) (num-exp 4))) 7)

(define (calc-de-un-numero-suelto)
  (calc 42))

(check-exn exn:fail? calc-de-un-numero-suelto)
```

- `check-equal?` recibe lo que se calcula y lo que debería dar. Cada uno es **una regla de la especificación**, escrita como código que se puede correr.
- `check-exn` verifica que algo *falla* como debe. Fíjate en que le pasamos el **nombre** del procedimiento, no una llamada, igual que `cuadrado` a `mapea` en la sección 2.
- `raco test archivo.rkt` las corre todas desde la terminal y dice cuántas pasaron y cuáles fallaron.

Nota Didáctica: qué es una prueba en este curso

Más adelante en el curso vas a escribir tus propias pruebas para el intérprete. La idea que conviene guardarse de esta lectura es esta: una prueba no es “probar a ver si funciona”. Una prueba es una regla de la especificación escrita de forma ejecutable. Si no puedes decir qué regla está verificando un `check-equal?`, probablemente no vale la pena tenerlo.

8. Nombres y with

Sesión 8

Esta sección es la preparación directa del ejercicio integrador. El enunciado completo, con lo que hay que entregar y cómo se califica, está en su propio documento. Aquí está el contenido.

8.1. La gramática completa

```
Exp ::= Num
      | Id                               nueva
      | add(Exp, Exp)
      | mul(Exp, Exp)
      | with Id = Exp in Exp             nueva
```

Tres producciones ya están resueltas desde la sección 4. Las dos nuevas traen algo que no habíamos visto: **nombres**.

8.2. Qué hace with

```
with y = add(3, 4) in mul(y, y)
```

1. Se calcula `add(3, 4)`. Da 7.
2. Dentro del cuerpo, `y` vale 7.
3. Se calcula el cuerpo, `mul(y, y)`, que ahora es `mul(7, 7)`. Da **49**.

Es lo que hace cualquier lenguaje cuando le pones nombre a un resultado intermedio. Aquí lo vamos a implementar nosotros.

8.3. Los structs que faltan

```
(struct id-exp (nombre) #:transparent)
```

```
(struct with-exp (nombre expr cuerpo) #:transparent)
```

- `id-exp` guarda un nombre: (`id-exp 'y`). La comilla simple dice que `y` es **un nombre del lenguaje que estamos representando**, no una variable de Racket.
- `with-exp` guarda tres cosas, en este orden: el nombre, la expresión que le da valor al nombre, y el cuerpo donde ese nombre se usa.

Una producción, un `struct`. La regla no cambió.

8.4. A calc le falta algo

```
> (calc (id-exp 'y))
```

¿Cuánto vale `y`? No se puede saber mirando el nodo: un nombre no trae su valor adentro. Depende de qué `with` lo haya ligado, más arriba en el árbol.

Entonces `calc` tiene que **ir cargando** las asociaciones nombre $\rightarrow$ valor mientras baja por el árbol. Un parámetro más, y eso es todo lo que cambia:

```
(define (calc e asocs)
  (match e
    [(num-exp n) n]
    [(id-exp nombre) (buscar nombre asocs)]
    [(add-exp e1 e2) (+ (calc e1 asocs) (calc e2 asocs))]
    [(mul-exp e1 e2) (* (calc e1 asocs) (calc e2 asocs))]
    ...))
```

- Los casos que ya tenías no miran `asocs`: solo lo pasan hacia abajo.
- `id-exp` es el único que lo *consulta*.
- Falta `with-exp`, que es el único que lo *cambia*.

8.5. Las asociaciones

Tres operaciones, y con eso basta:

```
(define (asocs-vacias) '())

(define (extender asocs nombre valor)
  (cons (cons nombre valor) asocs))

(define (buscar nombre asocs)
  (cdr (assoc nombre asocs)))
```

- `assoc` es de Racket: busca en una lista de pares el primero cuyo lado izquierdo es `nombre`, y devuelve el par completo. Por eso `buscar` le saca el `cdr`, que es el valor.
- Crear, extender, buscar. **Es una interfaz**, la misma idea del contador de la sección anterior.
- Que por dentro sea una lista de pares no le importa a `calc`.

8.6. `with-exp`

Tres preguntas, en orden:

1. **¿Qué hay que calcular primero?** El valor de `expr`, con las asociaciones que hay ahora.
2. **¿Con qué se extiende?** Con `nombre` ligado a ese valor.
3. **¿Dónde se evalúa el cuerpo?** Con las asociaciones **extendidas**, no con las de antes.

Y eso, escrito:

```
[(with-exp nombre expr cuerpo)
 (calc cuerpo (extender asocs nombre (calc expr asocs)))]
```

Léelo de adentro hacia afuera: `(calc expr asocs)` es el valor. `extender` arma las asociaciones nuevas con ese valor. Y el `(calc cuerpo ...)` de afuera evalúa el cuerpo con ellas.

8.7. La especificación completa

$$\begin{aligned} [\text{num}] \quad & (\text{calc } (\text{num-exp } n) \sigma) &= n \\ [\text{id}] \quad & (\text{calc } (\text{id-exp } x) \sigma) &= \text{lookup}(x, \sigma) \\ [\text{add}] \quad & (\text{calc } (\text{add-exp } e_1 e_2) \sigma) &= (\text{calc } e_1 \sigma) + (\text{calc } e_2 \sigma) \\ [\text{mul}] \quad & (\text{calc } (\text{mul-exp } e_1 e_2) \sigma) &= (\text{calc } e_1 \sigma) \times (\text{calc } e_2 \sigma) \\ [\text{with}] \quad & (\text{calc } (\text{with-exp } x e_1 e_2) \sigma) &= (\text{calc } e_2 \text{ extend}(\sigma, x, (\text{calc } e_1 \sigma))) \end{aligned}$$

- σ es lo que en el código se llama **asocs**. Misma cosa, otro nombre. Se usa la letra griega por costumbre del área, y porque en la especificación no estamos hablando de una lista sino de “lo que sabemos de los nombres”.
- **lookup** está programado en **buscar**. **extend** está programado en **extender**.
- Fíjate en la regla **[with]**: el σ con el que se evalúa el cuerpo **no** es el mismo con el que entró. Es la primera vez en el curso que el contexto de una evaluación cambia, y es exactamente lo que la unidad 1 va a formalizar.

8.8. Trazar con σ

Trazar **with** $y = \text{add}(3, 4)$ in $\text{mul}(y, y)$, empezando con σ_0 vacío. Abreviamos para que quepa: N3 y N4 son $(\text{num-exp } 3)$ y $(\text{num-exp } 4)$, A es $(\text{add-exp } N3 \ N4)$, y C es el cuerpo, $(\text{mul-exp } (\text{id-exp } y) \ (\text{id-exp } y))$.

$$\begin{aligned} & (\text{calc } (\text{with-exp } y \ A \ C) \ \sigma_0) \\ &= (\text{calc } C \ \text{extend}(\sigma_0, y, (\text{calc } A \ \sigma_0))) & [\text{with}] \\ &= (\text{calc } C \ \text{extend}(\sigma_0, y, (\text{calc } N3 \ \sigma_0) + (\text{calc } N4 \ \sigma_0))) & [\text{add}] \\ &= (\text{calc } C \ \text{extend}(\sigma_0, y, 3 + 4)) & [\text{num}] \\ &= (\text{calc } C \ \sigma_1) \quad \text{con } \sigma_1 = \text{extend}(\sigma_0, y, 7) \\ &= (\text{calc } (\text{id-exp } y) \ \sigma_1) \times (\text{calc } (\text{id-exp } y) \ \sigma_1) & [\text{mul}] \\ &= \text{lookup}(y, \sigma_1) \times \text{lookup}(y, \sigma_1) & [\text{id}] \\ &= 7 \times 7 \\ &= 49 \end{aligned}$$

Un renglón por paso, y una sola regla por renglón: no se juntan dos reglas distintas en el mismo paso. Abreviar una subexpresión con una letra está permitido, siempre que digas qué es. Lo que no se puede saltar es el paso donde σ se extiende: es justo lo que el trazado tiene que mostrar.

A. Referencia rápida

Todo lo que la unidad usa, con un ejemplo mínimo. Es lo que conviene tener a mano mientras resuelves algo.

Definir y decidir

```

(define pi 3.1416)
(define (f x) (* x x))
(if (< x 0) (- x) x)
(cond [(< n 0) -1] [else 1])

```

nombre para un valor
 nombre para una función
 dos casos, produce un valor
 varios casos, de arriba abajo

Números y cadenas

```

(- x)
(max 3 5)
(number->string 3)
(string-append "a" "b" "c")

```

el negativo de `x`, con dos argumentos es la resta
 el mayor de dos: 5
 de número a cadena: "3"
 pega cadenas: "abc"

Listas

```

'()
(list 1 2 3)
(cons 1 '(2 3))
(car '(1 2 3))
(cdr '(1 2 3))
(null? lst)
(length lst)
(assoc 'y lst)

```

la lista vacía
 construir una lista
 pegar al frente, devuelve una nueva
 el primer elemento: 1, también `first`
 el resto, como lista: '(2 3), también `rest`
 ¿está vacía? También `empty?`
 cuántos elementos
 busca el par cuyo lado izquierdo es 'y

Datos en variantes

```

(struct circulo (radio) #:transparent)
(circulo 3)
(circulo-radio c)
(circulo? c)
(match f [(circulo r) ...])
[(rectangulo b _) b]

```

define constructor, selectores y predicado
 construir uno
 sacar un campo
 ¿es de esta variante?
 un caso por variante
 _ ignora un campo

Módulos, se leen, no se escriben todavía

```

(provide f g)
(contract-out [f (-> any/c number?)])

```

qué se ve desde afuera
 qué promete una operación

Lo que la sección 7 deja como lectura adelantada, `parser-tools` y `RackUnit`, no está en esta referencia: la unidad no lo usa.

B. Errores frecuentes y cómo leerlos

Todos estos salen de código que aparece en estas notas. Racket es más explícito que la mayoría de los lenguajes: casi siempre el mensaje dice exactamente qué pasó.

arity mismatch: número de argumentos equivocado.

```
> (circulo 3 4)
circulo: arity mismatch;
  the expected number of arguments does not match the given number
  expected: 1
  given: 2
```

Le diste dos argumentos a algo que espera uno. Sale igual con funciones que con constructores de **struct**. Revisa la definición: *arity* es el número de argumentos.

no matching clause: a *match* le falta un caso.

```
> (area (cuadrado 4))
match: no matching clause for (cuadrado 4)
```

Definiste el **struct** y olvidaste el caso. El mensaje te dice qué valor llegó sin que ningún patrón lo reconociera. Ahí está la variante que falta.

Sale el mismo mensaje cuando el valor no es de ninguna de las variantes. (**area** 5) contesta **match: no matching clause for 5**: un *match* sin caso **else** rechaza lo que no reconoce, y ese rechazo es el que en la primera versión de **area** había que escribir a mano con **error**.

application: not a procedure: casi siempre, un paréntesis de más.

```
> ((+ 1 2))
application: not a procedure;
  expected a procedure that can be applied to arguments
  given: 3
```

Los paréntesis de afuera dicen “aplica lo de adentro”. Lo de adentro dio 3, y 3 no es una función. Cuenta los paréntesis.

contract violation: un argumento del tipo equivocado.

```
> (car '())
car: contract violation
  expected: pair?
  given: '()
```

Este en particular es el más común de todos: le pediste el primer elemento a una lista vacía. Suele significar que a una función recursiva le falta el caso base, o que el caso base está después de usar `car`.

Un **struct** que imprime raro: falta `#:transparent`.

```
> (opaco 5)
#<opaco>
```

No es un error, pero hace que sea difícil depurar: no ves nada de lo que hay adentro. Agrega `#:transparent` a la definición.

Sugerencia: cómo leer cualquier error de Racket

Los mensajes tienen siempre la misma forma: primero **quién** se quejó (`car:`, `circulo:`, `match:`), luego **qué esperaba** (`expected:`) y luego **qué recibió** (`given:`). Esas tres piezas suelen bastar. Empieza siempre por el nombre de la izquierda: te dice en qué función mirar.

C. Mapa de contenidos

Qué sección cubre cada elemento del programa de la unidad.

Conocimientos

Lo que se espera que tengas claro conceptualmente al cerrar la unidad. Son ideas que tienes que poder explicar, no cosas que tengas que producir.

Qué	Sección
Racket es de la familia Scheme: paréntesis, notación prefija, todo es una expresión que produce un valor	1
Las funciones son valores: se pasan, se devuelven, se guardan	2
La recursión es el mecanismo primario de repetición	2
struct define un tipo con campos, match despacha por variante	3
BNF es notación para gramáticas libres de contexto	4
Cada producción es un struct , y un programa un árbol de structs	4, 5, 8
Sintaxis concreta vs. sintaxis abstracta	4
Abstracción de datos: interfaz vs. implementación	7
Notación de especificación por casos	6
Módulos y contratos	7

Lo que la sección 7 dice sobre `parser-tools` y sobre `RackUnit` no está en esta lista a propósito. Es lectura adelantada, y se trabaja más adelante en el curso.

Habilidades

Lo que se espera que puedas trabajar por tu cuenta al cerrar la unidad. Aquí no basta con explicarlo: hay que poder hacerlo sentado frente al editor.

Qué	Sección
Escribir funciones recursivas sobre números y listas	2
Definir tipos con <code>struct</code> y crear instancias	3
Usar <code>match</code> para despachar por variante	3
Dada una gramática, definir los <code>structs</code> correspondientes	4, 8
Escribir una función que procese datos definidos por una gramática	5, 8
Leer una especificación y trazar el resultado para una entrada	6, 8
Organizar código en módulos y usar contratos básicos	7

La última está al nivel de *introducida*: la viste funcionar y sabes leerla. Producirla por tu cuenta es trabajo de las unidades siguientes.

Para seguir leyendo

El libro de texto del curso es **EOPL**, de Friedman y Wand, *Essentials of Programming Languages*, 3.^a edición. Las secciones que corresponden a esta unidad:

EOPL	Corresponde a
§1.1 <i>Recursively Specified Data</i>	secciones 2 y 4 de estas notas
§2.1–2.2 <i>Specifying Data via Interfaces</i>	sección 7
§2.5 <i>Abstract Syntax and Its Representation</i>	sección 4
§3.1–3.2 <i>Specification and Implementation</i>	sección 6

Dos advertencias al abrirlo. EOPL usa `define-datatype` donde nosotros usamos `struct` y `match`: la idea es la misma y la sintaxis no. Y su notación para especificaciones es ligeramente distinta de la nuestra: nosotros etiquetamos cada paso de un trazado con la regla que se aplicó, entre corchetes, y el libro no.